数学与 CS 问答笔记

answer-note

按日归档的问答笔记：三问反思法（关键点/变式/同类归类）+ 讲课叙事流 + CeTZ 图示，结构与课程笔记同构。

October 15, 2026

Contents

1. 当日起点：项目状态与工具状态 3
2. 在 `kv.go` 里落地 TDD：红-绿-重构的完整一轮 4
   - 2.1. 费曼直觉 4
   - 2.2. 硬核机制 4
     - 2.2.1. 三步循环、出口条件，一张图 4
     - 2.2.2. 你其实已经站在循环里：测试先行早已完成（实测） 5
     - 2.2.3. 沙盒复刻：三阶段实测（预演你接下来的两步） 6
     - 2.2.4. 两种“红”的对照：它其实在给你配音 8
     - 2.2.5. 之后每一轮怎么开：以 `Set` 的 `updated` 语义为例 8
     - 2.2.6. TDD 循环里的那条命令（以及它为什么偶尔“装死”） 10
     - 2.2.7. `assert` 还是 `require`：失败之后要不要继续 11
   - 2.3. 陷阱与易错 12
   - 2.4. 关键点 13
   - 2.5. 变式检验 13
   - 2.6. 同类归类 13
   - 2.7. Go 语言机制补充 13
     - 2.7.1. `go test` 的执行模型（`go help test` 原文） 13
     - 2.7.2. 白盒测试：为什么 `kv_test.go` 里写 `KV{}` 不用 import 14
     - 2.7.3. `testing.T` 的最小方法集（循环里够用了） 14
     - 2.7.4. 为什么 `require` 必须在测试自己的 goroutine 里用 14
   - 2.8. 本次操作与当前状态 15

Index of Figures 16

Index of Listings 16

1. 当日起点：项目状态与工具状态

距 10-13 起已经两天没有动代码。今天动手前的状态先钉在这里（它就是本次问答的“题干”）：

项目项	当前状态
`internal/db/kv.go`	`Open` / `Close` / `Get` / `Set` 已有实现；`Del` 只有签名（第 29 行）—— `missing function body`，整个包编译不过
`internal/db/kv_test.go`	2026-10-09 从 `test/` 迁来，与实现同包（白盒测试）；它的断言**比实现先写好**，正处在“测试已先行”的状态
`scratch/kvprobe/`	import 了 `internal/db`，被 `Del` 的缺失拖住——补完 `Del` 就能跑
测试工具链	`go test ./internal/db/` 可用；`go vet ./internal/db/` **测不出**这个错（下面有实测）；`go build ./...` 不编译 `_test.go`

今天的问题：

> 📌 **核心要点 如果我要按测试驱动开发（Test-Driven Development, TDD）的方式继续写，在 `kv.go` 中应该怎么做？**

一句话回应：你不需要“从零开始 TDD”——**考卷（测试）已经先写好，而且已经是红的。**你要做的是把红-绿-重构（Red-Green-Refactor）这个循环走完：先补 `Del` 让红精确地显示出来，再按测试的期望修 `Get`，之后每一轮小改动都重复同样的循环。09-30 那一节《用测试驱动开发，Go 的目录结构怎么摆》回答的是“测试放在哪里”；今天这节回答“循环怎么转”。

2. 在 `kv.go` 里落地 TDD：红-绿-重构的完整一轮

2.1. 费曼直觉

TDD 一句话：**先写考卷，再写答案**。

普通开发的顺序是“写完实现 → 补测试”；TDD 把顺序倒过来：

1. **红**：先写一个**会失败**的测试（它描述“代码应该做到、但现在还做不到”的事）；
2. **绿**：写**最小**的代码让它通过；
3. **重构**：在测试保护下整理代码（改名、去重复、提速），行为不许变。

为什么这对你现在的 `kv.go` 特别合适：

1. 你正卡着的两个问题——“`Del` 的 `deleted` 到底表示什么”、“`Get` 未命中算不算错误”——都是**语义（semantics）问题**。语义没法从代码里“长”出来，只能先定；TDD 的招式就是把语义写成测试，**测试通过 = 语义被满足**；
2. 你后面还要写 `cell.go`、`table.go`、`sorted_file.go`。这些模块边界情况很多（空、单元素、重复键），每加一个功能都有一套测试兜底，就等于手里始终有一张**回归网（regression net）**——改了 A 不怕悄悄碰坏 B。

> 💡 **背景与类比** 验收清单类比：装修是“先和施工队约定验收标准（插座通电、水压正常、地砖空鼓率），再开工”。测试就是这张验收清单的**可执行版本**——不需要有人念，机器每次替你念一遍。

2.2. 硬核机制

2.2.1. 三步循环、出口条件，一张图

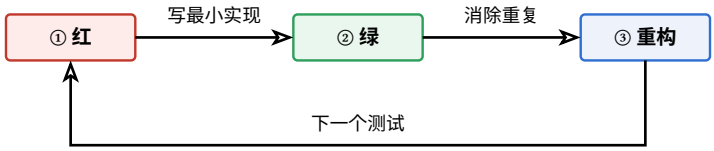


Figure 1: TDD 循环（CeTZ 自绘）：箭头上标的是“这一步要做的动作”——红节点里写一个失败测试，①→② 之间写最小实现，②→③ 之间消除重复，③ 绕回 ① 开始下一个测试

每一步的出口条件（“怎么算这一步走完了”）：

步	你敲什么	出口条件（严格版）
① 红	在 `_test.go` 里加一条新断言	`go test` 里这个测试**失败**，且失败原因是“功能还没写”，不是笔误或类型错
② 绿	在 `kv.go` 里写**最小**实现	同一条命令输出 `ok`（`PASS`）
③ 重构	不改行为，只改结构（改名、去重复、加接口）	再跑一次，仍然 `ok`——**绿进绿出**

三条规矩，每条都配了理由：

1. **红必须亲眼看见。**一个从没红过的测试，不能证明它会红——它可能因为函数名写错被静默忽略、可能这个功能你早就写好了。你分不清“测试通过”和“测试根本没跑”；
2. **绿要最小。**只写“刚好让它通过”的代码，多写的那部分没有测试支撑，等于又回到“先写实现后补测试”；
3. **一次只处理一个失败。**五个测试同时红，你改的一行到底修好了哪个？一次一个，红得越精确越省时间。

2.2.2. 你其实已经站在循环里：测试先行早已完成（实测）

你现在的处境不是“从零 TDD”。打开 `kv_test.go` ——考卷比答案先写好了。它有三组断言，分别对应你三个方法的契约（contract）：

```
1   func TestKVBasic(t *testing.T) {
2     kv := KV{}
3     err := kv.Open()
4     assert.Nil(t, err)
5     defer kv.Close()
6
7     updated, err := kv.Set([]byte("k1"), []byte("v1"))
8     assert.True(t, updated && err == nil)
9
10    val, ok, err := kv.Get([]byte("k1"))
11    assert.True(t, string(val) == "v1" && ok && err == nil)
12
13    _, ok, err = kv.Get([]byte("xxx"))
14    assert.True(t, !ok && err == nil) // ← 第 22 行：未命中要 ok=false 且 err=nil
15
16    updated, err = kv.Del([]byte("xxx"))
17    assert.True(t, !updated && err == nil) // 删不存在的键：deleted=false
18
19    updated, err = kv.Del([]byte("k1"))
20    assert.True(t, updated && err == nil) // 删存在的键：deleted=true
21
22    _, ok, err = kv.Get([]byte("xxx"))
23    assert.True(t, !ok && err == nil) // ← 第 31 行：删掉之后又是“未命中”
24  }
```

Listing 1: `internal/db/kv_test.go` 全文（第 9 行起）：三组契约——`Set` 后 `Get` 命中、`Get` 未命中用 `!ok && err == nil` 表达、`Del` 用 `updated` 表达“删没删到”；第 22、31 行是后面两轮红的关键行

对照实现，三类断言各有命运：

测试行	契约	实现现状
第 15–19 行	`Set` 后 `Get` 应能拿回值	已满足（`Set` / `Get` 的实现正确）

第 24–28 行	`Del` 删不存在的键返回 `false`、删存在的键返回 `true`	**未实现**——`Del` 只有签名，包都编译不过
第 22、31 行	`Get` 未命中应是 `!ok && err == nil`	**冲突**——实现返回 `errors.New(..)` 与期望相反

于是“红”早已就位——只是被编译错误压在下面。先让它显示出来（仓库原样的实测）：

```
$ go vet ./internal/db/
 （无输出，退出码 0 —— 看起来"通过"，但这不算数）

$ go build ./...
# github.com/zz-cpp/db-go/internal/db
internal/db/kv.go:29:6: missing function body

$ go test ./internal/db/ -run TestKVBasic -v
internal/db/kv.go:29:6: missing function body
FAIL  github.com/zz-cpp/db-go/internal/db [build failed]
FAIL
```

Listing 2: 仓库现状实测：编译器知道 `Del` 缺函数体（第 29 行），`go test` 报的是 build failed——这是“红”，只是红在测试跑起来之前

> ⚠️ **陷阱预警** **`go vet` 在这里骗你**——它对缺函数体的声明**静默通过**。用一个最小对照包实测过：单独一个文件只写 `func Missing() int`（无函数体），`go vet ./vetprobe` 无输出、退出码 0；`go build ./vetprobe` 立刻报 `vetprobe/x.go:3:6: missing function body`。所以 TDD 的红灯信号只能用 `go test`（或 `go build`），**不能用** **`go vet`**——这与 09-30《go vet 通过 ≠ 能编译》的结论一致，只是这次连“被点名的包”也不例外。

2.2.3. 沙盒复刻：三阶段实测（预演你接下来的两步）

为了不动你的文件，我把 `kv.go` 与 `kv_test.go` 原样复制到一个独立沙盒（`$TMPDIR/kvtdd`、模块名 `kvtdd`），在那里把循环走完。下面三阶段就是“你接下来两步会发生什么”的预演。

阶段 1（原样不动）：编译型红。

```
$ go test ./...
# kvtdd [kvtdd.test]
./kv.go:29:6: missing function body
FAIL  kvtdd [build failed]
FAIL
```

Listing 3: 阶段 1：包编译不过，测试没有资格跑——“红”的第一形态

阶段 2（只补 `Del`）：断言型红。

```
1  func (kv *KV) Del(key []byte) (deleted bool, err error) {
2    _, ok := kv.mem[string(key)]
3    delete(kv.mem, string(key))
4    return ok, nil
5  }
```

Listing 4: `Del` 的“先查后删”实现：`delete` 内建函数对不存在的键是安全空操作、且不返回任何信息，所以“删没删到”只能自己先查一遍——这正是测试第 24–28 行逼出来的设计

```
1  $ go test ./...
2  --- FAIL: TestKVBasic (0.00s)
3      kv_test.go:22: Error: Should be true
4      kv_test.go:31: Error: Should be true
5  FAIL
6  FAIL  kvtdd 0.003s
7  FAIL
```

Listing 5: 阶段 2：包能编译、测试能跑了——失败点精确落在 `kv_test.go` 第 22、31 行，正是两处 `Get` 未命中断言（为版面计，Error Trace 细行已省略）

注意这次红得多**精确**：它不说“`Del` 有问题”，它说“你离全绿只差 `Get` 这一条契约”。这就是“红”的价值——**越精确的红，越接近下一行该写的代码。**

阶段 3（按测试期望改 `Get` 未命中）：绿。

```
1  func (kv *KV) Get(key []byte) (val []byte, ok bool, err error) {
2    val = kv.mem[string(key)]
3    if val != nil {
4      return val, true, nil
5    }
6    return nil, false, nil
7  }
```

Listing 6: `Get` 的未命中分支：`return nil, false, nil`——用 `ok` 表达“没找到”，而不是 `error`（10-13 已经裁决过的语义）；同时把第 3 行不再使用的 `import "errors"` 删掉，否则编译器报 imported and not used

```
1  $ go test -v ./...
2  === RUN   TestKVBasic
3  --- PASS: TestKVBasic (0.00s)
4  PASS
5  ok    kvtdd 0.003s
```

Listing 7: 阶段 3：绿。到这里循环转完一圈：红 → 绿；这一步改动太小，没有重构空间——重构不是必做动作，是“绿灯下顺手擦一擦”

把上面两步落回你的仓库，就是（也是本轮结束时“还欠着”的全部工作）：

位置	现在	改成
`kv.go` 第 29 行 `Del`	只有签名（缺函数体）	先查后删的 4 行实现（见阶段 2 代码）

`kv.go` 第 21 行 `Get` 未命中分支	`return val, ok, errors.New("the key is not existed!")`	`return nil, false, nil`，并删掉第 3 行 `import "errors"`
验收	`go test ./...` 报 `FAIL`（build failed）	`go test ./...` 输出 `ok`；`go run ./scratch/kvprobe` 跟着跑通

2.2.4. 两种“红”的对照：它其实在给你配音

红的形态	什么时候出现	实测长什么样 / 含义
编译型红	引用了还不存在的函数或名字、语法错	`./kv.go:29:6: missing function body` + `[build failed]`。测试**没资格跑**；先让编译过，红才有信息量
断言型红	代码能跑，但行为与期望不符	`--- FAIL: TestKVBasic` + `kv_test.go:22: Should be true`。这才是真正的红灯：告诉你哪条契约没兑现
假的绿（最危险）	测试名写错、跑错包、被缓存糊弄	输出 `ok` 或 `[no tests to run]`。你以为在 TDD，其实测试没跑——症状是“它永远通过”（测试函数名必须 `TestXxx`，09-30 有实测）

2.2.5. 之后每一轮怎么开：以 `Set` 的 `updated` 语义为例

循环的下一轮入口，永远是一个“实现里没有定义清楚的行为”。你手里现成的一个：`Set` 永远返回 `true` —— `updated` 的语义其实从没被定义过（写新键算“更新”吗？覆盖旧键算吗？）。

TDD 的做法是：**先把语义写成测试，再让实现跟上**。但注意这两种候选语义有本质区别：

候选语义	删测试的“红/绿”命运	言会怎么写

`updated` = “写入成功”（恒为 true）	辨在**已经是绿**——这个测试永远不会红，信息量最低 次 就 `Set` ； `assert.True` 覆 盖 时 也 `assert.True`
`updated` = “这次覆盖了一个已存在的键”	辨在**必红**（实现恒 true）→ 改实现 → 绿。这才是有分辨力的测试 次 期 `Set` 望 ； `false` 第 二 次 （覆 盖） 期 望 `true`

> 📌 **核心要点** 选哪种语义是**设计决策**，TDD 替你做不了——得看讲义怎么规定、上层调用者怎么用这个返回值。TDD 能做的，是把你的决定**钉进测试里**：从此以后，谁想悄悄改这个语义，测试会先红。

下一张“考卷”的样子（表驱动测试（table-driven test）版本，风格与 09-30 的示例一致）：

```
1   func TestKVSetUpdated(t *testing.T) {
2     tests := []struct {
3       name   string
4       preset bool // 是否先把这个键写一次
5       want   bool
6     }{
7       {"新建键 -> updated=false", false, false},
8       {"覆盖已有键 -> updated=true", true, true},
9     }
10
11    for _, tt := range tests {
12      t.Run(tt.name, func(t *testing.T) {
13        kv := &KV{}
14        if err := kv.Open(); err != nil {
15          t.Fatal(err)
16        }
17        if tt.preset {
18          kv.Set([]byte("k1"), []byte("v1"))
19        }
20        updated, err := kv.Set([]byte("k1"), []byte("v2"))
21        assert.NoError(t, err)
22        assert.Equal(t, tt.want, updated)
23      })
24    }
25  }
```

Listing 8: 下一轮的表驱动考卷：按“覆盖”语义写，跑起来必红（红在“新建键”那行）；语义定了再写实现——先别急着抄实现，红过之后再写

对应地，实现会变成（同样先看红，再写）：

```
1   func (kv *KV) Set(key []byte, val []byte) (updated bool, err error) {
2     _, updated = kv.mem[string(key)]
3     kv.mem[string(key)] = val
4     return updated, nil
5   }
```

Listing 9: 按“覆盖”语义改写 `Set`：先查旧值决定 `updated`，再写入

⚠ **陷阱预警** 改语义是**全局影响**：一旦按“覆盖”语义改掉 `Set`，现有测试第 16 行（第一次写新键却断言 `updated == true`）会立刻变红——那时你要做的是让“新测试、老测试、实现”三方对齐（通常是同时改老测试）。这就是“测试即契约”的代价与好处：任何语义改动都会被逼着显式处理。

2.2.6. TDD 循环里的那条命令（以及它为什么偶尔“装死”）

循环里最常用的一条：

```
1   $ go test -v -count=1 ./internal/db/
```

Listing 10: `-v` 让你看见每个测试的开始/结果；`-count=1` 禁用缓存、强制真跑。习惯了就别省这两个参数

为什么老手坚持带 `-count=1` ——实测（沙盒）：

```
$ go test ./...            # 第一次：真跑
ok    kvtdd 0.003s

$ go test ./...            # 立刻再跑：命中缓存
ok    kvtdd (cached)

$ go test -count=1 ./...   # 强制真跑
ok    kvtdd 0.004s

$ go test                  # 不带包参数（本目录模式）
PASS
ok    kvtdd 0.003s
```

Listing 11: `(cached)` 不是“刚跑过”，而是“上次成功过、且包与依赖没变”；本目录模式和包列表模式的缓存行为不同（见下）

出处是 `go help test` 原文（两段）：

```
In package list mode only, go test caches successful package test
results to avoid unnecessary repeated running of tests. ...
When this happens, go test prints '(cached)' in place of the
elapsed time in the summary line.
...
The idiomatic way to disable test caching explicitly is to use -count=1.
```

Listing 12: `go help test` 关于缓存的原文：只在“包列表模式”（如 `go test ./...`）有缓存；`-count=1` 是官方钦定的关缓存方式（引文中 ... 表示原文相隔数行）

```
The first, called local directory mode, occurs when go test is
invoked with no package arguments (for example, 'go test' or 'go
test -v'). ... In this mode, caching (discussed below) is disabled.
```

Listing 13: 同一个帮助的另一段：不带包参数时是“本地目录模式”，缓存直接关闭——所以你在 `internal/db/` 里敲裸 `go test` 永远是真实执行（引文中 ... 表示原文相隔数行）

规律总结成一句话：**在本目录敲 `go test` 没有缓存；敲 `go test ./...`（包列表模式）会有缓存，能不能信就看 `(cached)` 字样；要“确定真跑”就用 `-count=1`。** 缓存本身基于“包内容 + 依赖 + 可缓存 flag”的哈希——正常 TDD 里你改完代码，缓存自然失效，不会骗你；骗人的场景只有一个：你改了文件却看到 `cached`（说明改的不是被编译的那个文件/包）。

2.2.7. `assert` 还是 `require`：失败之后要不要继续

你现在的测试通篇用 `assert.True(...)`。它有个关键行为：**失败之后测试会继续跑**。出处（本机 `go doc` 原文）：

```
// assert (go doc github.com/stretchr/testify/assert 原文)
All functions in this package return a bool value indicating whether the
assertion has passed.

// require (go doc github.com/stretchr/testify/require 原文)
Package require implements the same assertions as the assert package but
stops test execution when a test fails.
```

Listing 14: `assert`：失败只记录、返回 false；`require`：失败即停（内部调用 `testing.T.FailNow`）

包	失败之后	适合放在哪
`assert`	记录失败，**继续**执行后面的断言	互相独立的检查点：值对不对、`ok` 对不对、`err` 空不空
`require`	立即中止本测试（`FailNow`）	**前置条件**：`Open` 失败、关键 `err != nil`——后面已经没法在有效状态下继续了

> 💡 **背景与类比** 一个具体场景：如果 `Open()` 就失败了，后面所有 `Set` / `Get` 都在垃圾状态下跑，会炸出一串连锁报错——根因只有一个。这种“不满足就别往下走”的检查，用 `require.NoError(t, err)`；后续行为断言继续用 `assert`。

（顺带：`assert` 的函数都返回 bool，所以 `if !assert.Equal(...) { return }` 是“手搓 require”的最小形态——机制就这点，不必觉得神秘。）

2.3. 陷阱与易错

1. **没看见红就写实现**。那不是 TDD，是“给已完成的实现补测试”。补测试没错，但别在“我确定它会红”上省掉那一次运行——那是唯一能证明测试有分辨力的动作；
2. **拿 `go vet` 当红灯信号**。今天实测反例：它对 `missing function body` 静默通过（`go build` 才报）。红灯信号只认 `go test` / `go build`；
3. **把 `(cached)` 当“刚跑过”**。`-count=1` 是最便宜的保险；看到 `cached` 就默认“这次没跑”；
4. **合体断言不好查**：`assert.True(t, !ok && err == nil)` 失败时只报 `Should be true`——你看不出是 `ok` 错还是 `err` 错。要么拆成两条断言，要么加消息：`assert.True(t, !ok && err == nil, "未命中应为 ok=false 且 err=nil")`；
5. **测试之间共享状态**。每个测试都自己 `kv := KV{}` + `Open()`；不要用包级变量、不要在测试之间传 map——一个测试的残留会让另一个红/绿得莫名其妙；
6. **一次写一大坨测试**。五个断言一起红，你分不清先修哪个，也看不出每一步的进展；
7. **测试名首字母小写**（`testKVBasic`）。不报错、不警告、不影响退出码，只是**静默不跑**——“测试永远通过”的第一嫌疑人（09-30 有实测）。

> ⚠️ **陷阱预警 “确认变绿”的标准动作**：`-v` 输出里亲眼看到 `--- PASS: 测试名`，并用 `-count=1` 保证真跑。只看到 `ok` 两个字，可能只是缓存；只看到没有报错，可能测试压根没被收集。

2.4. 关键点

> 📌 **核心要点** TDD 在 Go 里就是一条命令的循环：**改 `_test.go`（写期望）→ `go test -v -count=1 ./internal/db/` 看红（且因对的原因红）→ 改 `kv.go`（最小实现）→ 同一条命令看绿 → 需要时在绿灯下重构**。你离第一次全绿只差两个补丁：`Del` 补上函数体；`Get` 未命中改 `return nil, false, nil`（并删 `import "errors"`）。

1. 测试的价值不在“证明现在对”，而在“钉住将来”——它是一份自动执行的契约；改坏了，它先叫；
2. 你的起点比标准 TDD 还省事：**考卷已经写好了**（`kv_test.go`），红也已经在位——别人要从“想清楚该断言什么”开始，你从“让红灯显示、再点绿”开始；
3. 循环的入口永远是一个“实现里没定义清楚的行为”：你手里现成的下一个是 `Set` 的 `updated` 语义。

2.5. 变式检验

1. **换成“别人报了个 bug”**：先写一个**复现 bug 的测试**（它红）→ 修实现（它绿）→ 让这个测试留下当回归网。“bug 驱动开发”是 TDD 的亲兄弟，流程完全一样；
2. **换成“开新文件 `cell.go`”**：先写 `cell_test.go` 的第一条（比如 `func TestCellEncodeDecode(t *testing.T)`，引用还不存在的 `Cell{}`）→ 编译型红 → 顺着编译器报的“缺哪个名字”挨个补。这正是 09-30 那节“`undefined: KV` 是起点”的重用；
3. **换成“重构”**：想给 `KV` 加锁、换数据结构、拆方法时，先让测试全绿，再动结构，再跑测试——**重构是绿进绿出**，不存在“重构成新功能”这回事；
4. **换成“没现成测试的新功能”**：流程一样，只是考卷要你自己从头写。写不出断言，通常说明需求还没想清楚——这本身就是 TDD 的早期警报。

2.6. 同类归类

1. 与 09-30《用测试驱动开发，Go 的目录结构怎么摆》配套：那节回答“测试放哪里、长什么样”（`_test.go`、同包、`TestXxx`），今天回答“循环怎么转”（红-绿-重构、命令、红的两种形态）；
2. 与 10-13《Go 有没有 assert》衔接：今天两处待办里的 `Get` 语义（用 `ok` 不用 `error`）就是那节“决策表”的落地；`assert` / `require` 的分工也延续那节的“检查手段分层”；
3. 外面的名字与谱系：Kent Beck 的 Red-Green-Refactor（测试驱动开发“教义”的来源）；Go 社区把它简化成“`go test` + 表驱动测试”，没有测试运行器（test runner）配置文件这一层；
4. 往下延伸：`sorted_file.go`、`merge.go` 这些模块的边界情形多（空文件、单元素、重复键、迭代器耗尽），TDD + 表驱动在那里的收益最大——每加一行用例，就永久买下一份保险。

2.7. Go 语言机制补充

2.7.1. `go test` 的执行模型（`go help test` 原文）

1. **构建方式**：“‘Go test’ recompiles each package along with any files with names matching the file pattern “`*_test.go`”.” —— 测试文件不参与 `go build`，只在跑测试时被拉进编译；

2. **每个包一个测试二进制**："Each listed package causes the execution of a separate test binary." —— 这也是"多包并行跑测试"的机制来源；
3. **构建期自带一轮 vet**："As part of building a test binary, go test runs go vet on the package and its test source files to identify significant problems. If go vet finds any problems, go test reports those and does not run the test binary." —— 所以你循环里不必单独跑 vet（要关用 `-vet=off`，要全开用 `-vet=all`）；
4. **测试的运行目录 = 包的源码目录**："When 'go test' runs a test binary, it does so from within the corresponding package's source code directory." —— 以后用相对路径读 `testdata/` 夹具时的前提。

2.7.2. 白盒测试：为什么 `kv_test.go` 里写 `KV{}` 不用 import

同目录、声明 `package db`（不带 `_test` 后缀）→ 测试与被测代码是**同一个包**：直接用 `KV{}`，甚至能摸到未导出的字段 `kv.mem`。这叫白盒测试（white-box test），你 10-09 把测试迁进 `internal/db/` 之后享受的就是这个待遇。另一种形态是 `package db_test`（外部测试包，黑盒）：必须 import、只能使用导出的 API——要验证"对外契约"时用它；两者可以在同一目录共存。

2.7.3. `testing.T` 的最小方法集（循环里够用了）

方法	失败时的行为	对应 TESTIFY
`t.Error` / `t.Errorf`	记录失败，**继续**跑	`assert.*`（软断言）
`t.Fatal` / `t.Fatalf`	记录失败并**中止本测试函数**（内部 `runtime.Goexit`）	`require.*`
`t.Log` / `t.Logf`	不算失败，只在 `-v` 下显示	—
`t.Run`	起一个子测试（subtest），返回是否通过	表驱动测试的骨架；`-run '父/子'` 可单独跑一个用例

出处：`go doc testing.T`、`go doc testing.T.Run`。看懂这张表就明白：`assert` ≈ `Error`、`require` ≈ `Fatal` 的包装——这就是上一节"失败之后要不要继续"取舍的语言层根据。

2.7.4. 为什么 `require` 必须在测试自己的 goroutine 里用

`go doc github.com/stretchr/testify/require` 原文："it must be called from the goroutine running the test function, not from other goroutines created during the test." 原因：`require` 靠 `FailNow` 中止，而 `FailNow` 内部是 `runtime.Goexit`——它只终止**调用它的那个 goroutine**

（呼应 10-13：“recover 只对当前 goroutine 有效”，`Goexit` 同理）。以后写并发测试时，goroutine 里的断言只能用 `assert`，再自己把结果同步回测试主 goroutine。

2.8. 本次操作与当前状态

1. 新增本文件 `note/2026-10-15.typ`（当天问答归档，编译产出 `note/2026-10-15.pdf`）；
2. 沙盒实验在 `$TMPDIR/kvtdd`（会话临时目录，**不入库**）：复制了 `kv.go` / `kv_test.go`（模块名改为 `kvtdd`），完成三阶段实测与缓存对照；其中 `_vetprobe/` 是“vet 对缺函数体静默”的最小对照实验；
3. 文件写入方式说明：本次会话的文件工具写范围不含本项目 `note/` 目录（只允许 `/tmp`、`/home/zz/note` 等），正文先在 `$TMPDIR` 写好、再经 shell 复制到 `note/`——若想以后直接用文件工具写入，可在 Settings > Sandbox > Also writable 里加上本目录；
4. **没有改动项目代码**——你的两条待办完好（也正是本轮循环的“红”）：
   1. `internal/db/kv.go` 第 29 行：补 `Del` 的函数体（先查后删，见“阶段 2”代码块）；
   2. `internal/db/kv.go` 第 21 行：`Get` 未命中改 `return nil, false, nil`，同时删第 3 行 `import "errors"`。

   做完之后 `go test ./...` 应输出 `ok`，`go run ./scratch/kvprobe` 也会跟着跑通——那是你的“实验区第一次全通”时刻；
5. 验证记录：`go vet ./internal/db/` 无输出（不代表通过）；`go build ./...` 报 `kv.go:29:6: missing function body`；沙盒三阶段与缓存对照见正文实测；本笔记经 `typst compile` 零错误；
6. 建议的下一步：按上面两条改掉 `kv.go`，跑 `go test -v -count=1 ./internal/db/`；然后决定 `Set` 的 `updated` 语义，把“下一张考卷”（见“以 `Set` 的 `updated` 语义为例”一节）写进 `kv_test.go`。

Index of Figures

Figure 1 TDD 循环（CeTZ 自绘）：箭头上标的是“这一步要做的动作”——红节点里写一个失败测试，①→② 之间写最小实现，②→③ 之间消除重复，③ 绕回 ① 开始下一个测试 4

Index of Listings

Listing 1 `internal/db/kv_test.go` 全文（第 9 行起）：三组契约—— `Set` 后 `Get` 命中、`Get` 未命中用 `!ok && err == nil` 表达、`Del` 用 `updated` 表达“删没删到”；第 22、31 行是后面两轮红的关键行 5

Listing 2 仓库现状实测：编译器知道 `Del` 缺函数体（第 29 行），`go test` 报的是 build failed——这是“红”，只是红在测试跑起来之前 6

Listing 3 阶段 1：包编译不过，测试没有资格跑——“红”的第一形态 6

Listing 4 `Del` 的“先查后删”实现：`delete` 内建函数对不存在的键是安全空操作、且不返回任何信息，所以“删没删到”只能自己先查一遍——这正是测试第 24–28 行逼出来的设计 7

Listing 5 阶段 2：包能编译、测试能跑了——失败点精确落在 `kv_test.go` 第 22、31 行，正是两处 `Get` 未命中断言（为版面计，Error Trace 细行已省略） 7

Listing 6 `Get` 的未命中分支：`return nil, false, nil` ——用 `ok` 表达“没找到”，而不是 `error`（10-13 已经裁决过的语义）；同时把第 3 行不再使用的 `import "errors"` 删掉，否则编译器报 imported and not used 7

Listing 7 阶段 3：绿。到这里循环转完一圈：红 → 绿；这一步改动太小，没有重构空间——重构不是必做动作，是“绿灯下顺手擦一擦” 7

Listing 8 下一轮的表驱动考卷：按“覆盖”语义写，跑起来必红（红在“新建键”那行）；语义定了再写实现——先别急着抄实现，红过之后再写 10

Listing 9 按“覆盖”语义改写 `Set`：先查旧值决定 `updated`，再写入 10

Listing 10 `-v` 让你看见每个测试的开始/结果；`-count=1` 禁用缓存、强制真跑。习惯了就别省这两个参数 10

Listing 11 `(cached)` 不是“刚跑过”，而是“上次成功过、且包与依赖没变”；本目录模式和包列表模式的缓存行为不同（见下） 11

Listing 12 `go help test` 关于缓存的原文：只在“包列表模式”（如 `go test ./...`）有缓存；`-count=1` 是官方钦定的关缓存方式（引文中 … 表示原文相隔数行） . 11

Listing 13 同一个帮助的另一段：不带包参数时是“本地目录模式”，缓存直接关闭——所以你在 `internal/db/` 里敲裸 `go test` 永远是真实执行（引文中 … 表示原文相隔数行） 11

Listing 14 `assert`：失败只记录、返回 false；`require`：失败即停（内部调用 `testing.T.FailNow`） 12